

```

#include <Wire.h>
#include <Adafruit_VL53L0X.h>
#include <IRremote.h>

const int leftencoderApin = 2;
const int leftencoderBpin = 3;
const int rightencoderApin = 18;
const int rightencoderBpin = 19;
const int leftdirpin = 9;
const int rightdirpin = 10;
const int leftpwmpin = 11;
const int rightpwmpin = 12;

int leftencoderpulse = 0;
int rightencoderpulse = 0;
int currentleftencoderpulse = 0;
int currentrightencoderpulse = 0;
// distance variables
double ipr = 18.85; // inches per revolution
double tpr = 900; // ticks per revolution
int targetangle = 0;
int pwm = 100; // pwm motor power
int offset = 5; // motor power offset
// PID variables
double error = 0; // set speed - actual speed
double eI = 0; // integral error
double eD = 0; // derivative error
double prev_error = 0;
const float Kp = 1.8; // proportional gain
const float Ki = 0.4; //integral gain
const float Kd = 0.05; // derivative gain
float omega = 0;
float vL = 0;
float vR = 0;
int i = 0; // for motor start delay
const float wheelbase = 13.7; // inches
bool runonce = false; // the run once bool for turning
// Gyro Variables
const int MPU_ADDR = 0x68;

```

```

const float GYRO_SCALE = 131.0; //  $\pm 250^\circ/\text{s}$  range
float yaw = 0;
float gyroZOffset = 0;
unsigned long prevTime = 0;
// IR Configuration
const int receiver = 8;          // IR receiver pin
IRrecv irrecv(receiver);        // Create IR receiver object
static uint32_t last_decodedRawData = 0; // Stores last received code
// State management
enum SystemState {
    OFF,
    STANDBY,
    MOVING,
    SEARCHING
};
SystemState currentState = OFF;
// ToF Variables
Adafruit_VL53L0X loxFarRight = Adafruit_VL53L0X(); // Pin 4
Adafruit_VL53L0X loxMidRight = Adafruit_VL53L0X(); // Pin 5
Adafruit_VL53L0X loxMidLeft = Adafruit_VL53L0X(); // Pin 6
Adafruit_VL53L0X loxFarLeft = Adafruit_VL53L0X(); // Pin 7

#define XSHUT_FAR_RIGHT 4
#define XSHUT_MID_RIGHT 5
#define XSHUT_MID_LEFT 6
#define XSHUT_FAR_LEFT 7
unsigned long lastToFRead = 0;

float distanceFarRight = 0;
float distanceMidRight = 0;
float distanceMidLeft = 0;
float distanceFarLeft = 0;
float distancethreshold = 200;
bool farRight = (distanceFarRight <= distancethreshold);
bool midRight = (distanceMidRight <= distancethreshold);
bool midLeft = (distanceMidLeft <= distancethreshold);
bool farLeft = (distanceFarLeft <= distancethreshold);

// Sound and Motion Variables

```

```

const int sensorAnalogPin = A0;    // Analog input
const int sensorDigitalPin = 28;   // Digital input D28
const int ledPin = LED_BUILTIN;    // Built-in LED
const int buzzerPin = 29;          // Buzzer output
unsigned long lastDisturbanceTime = 0;
const unsigned long debounceTime = 500; // 0.5s debounce

// LED Configuration
#define RED 30
#define GREEN 31
#define BLUE 32

// Search mode variables
bool disturbanceDetected = false;
int flashState = 0;
const long beepInterval = 500;    // 500ms beep/flash interval

// Sound detection settings
unsigned long lastNoiseTime = 0;
const unsigned long noiseCooldown = 1000;
int loudNoiseThreshold;
int quietLevel;

unsigned long startMillis;
unsigned long currentMillis;
unsigned long detectionDuration = 10000; // 10 seconds in milliseconds

// PIR Sensor Pins (Inputs)
int pirPin1 = 22; // HC-SR501 #1
int pirPin2 = 23; // HC-SR501 #2
int pirPin3 = 24; // HC-SR501 #3
int pirPin4 = 25; // HC-SR501 #4

// Motion detection settings
unsigned long lastMotionTime = 0;
const unsigned long motionCooldown = 1000;

/***** Start of Setup *****/

```

```

void setup()
{

    Serial.begin(115200);
    Wire.begin();
    pinMode(leftencoderApin, INPUT);
    pinMode(leftencoderBpin, INPUT);
    // Left encoder interrupts
    attachInterrupt(digitalPinToInterrupt(leftencoderApin), ALeadingLeft, RISING);
    attachInterrupt(digitalPinToInterrupt(leftencoderBpin), BLeadingLeft, RISING);
    // Right encoder interrupts
    attachInterrupt(digitalPinToInterrupt(rightencoderApin), ALeadingRight,
RISING);
    attachInterrupt(digitalPinToInterrupt(rightencoderBpin), BLeadingRight,
RISING);
    pinMode(leftdirpin, OUTPUT);
    pinMode(rightdirpin, OUTPUT);
    pinMode(leftpwmpin, OUTPUT);
    pinMode(rightpwmpin, OUTPUT);
    analogWrite(leftpwmpin, 0);
    analogWrite(rightpwmpin, 0);

    pinMode(RED, OUTPUT);
    pinMode(GREEN, OUTPUT);
    pinMode(BLUE, OUTPUT);
    digitalWrite(RED, HIGH);
    digitalWrite(GREEN, LOW);
    digitalWrite(BLUE, LOW);
    // Gyro setup (Initialize MPU)
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(0x6B); // PWR_MGMT_1
    Wire.write(0);     // Wake up
    Wire.endTransmission();
    calibrateGyro();
    prevTime = micros();
    // Initialize ToF sensors
    pinMode(XSHUT_FAR_RIGHT, OUTPUT);
    pinMode(XSHUT_MID_RIGHT, OUTPUT);
    pinMode(XSHUT_MID_LEFT, OUTPUT);

```

```

pinMode(XSHUT_FAR_LEFT, OUTPUT);

initializeSensor(loxFarRight, XSHUT_FAR_RIGHT, 0x30, "Far Right");
initializeSensor(loxMidRight, XSHUT_MID_RIGHT, 0x31, "Middle Right");
initializeSensor(loxMidLeft, XSHUT_MID_LEFT, 0x32, "Middle Left");
initializeSensor(loxFarLeft, XSHUT_FAR_LEFT, 0x33, "Far Left");
// Initialize sound/motion sensors
pinMode(sensorDigitalPin, INPUT);
pinMode(ledPin, OUTPUT);
pinMode(pirPin1, INPUT);
pinMode(pirPin2, INPUT);
pinMode(pirPin3, INPUT);
pinMode(pirPin4, INPUT);
pinMode(buzzerPin, OUTPUT);
// IR initialization
irrecv.enableIRIn(); // Start the IR receiver
calibrateSensor();
Serial.println("System initialization complete!");
delay(200); // Allows the serial monitor to connect before robot starts
}

void loop()
{
    // Check for IR input
    if (irrecv.decode())
    {
        translateIR();
        irrecv.resume();
    }
    readAllToFensors(); // constantly checks values of ToF sensors to see if
something is within range
    if(currentState == OFF) {
        stopMotors();
        digitalWrite(RED, HIGH);
        digitalWrite(GREEN, LOW);
        digitalWrite(BLUE, LOW);
        digitalWrite(buzzerPin, LOW);
    }
}

```

```

if(currentState == MOVING) {
    digitalWrite(buzzerPin, LOW);
    if(checkFrontObstacle()) // OBSTACLE CASE
    {
        digitalWrite(buzzerPin, LOW);
        double randomTurn = random(100, 260); // generate random angle to turn to
        Serial.println("Object Detected");
        readAllToF Sensors();
        runonceturn = true; // prepare to turn
        stopMotors();
        targetangle = yaw + randomTurn;

        delay(1000);
        // starts turning and updating gyro/pid as it turns
        while(runonceturn == true)
        {
            updateGyro();
            computePID();
            turnRight();
        }
    } else if ((leftencoderpulse < (currentleftencoderpulse + 84*(tpr/ipr))) &&
(rightencoderpulse < (currentrightencoderpulse + 84*(tpr/ipr))))// left encoder
pulse and right in coder pulse less than current left encoder pulse plus 5ft and
current right encoder pulse pluse 5ft
    {
        // NORMAL CASE | Green (G = HIGH)
        digitalWrite(RED, LOW);
        digitalWrite(GREEN, HIGH);
        digitalWrite(BLUE, LOW);
        digitalWrite(buzzerPin, LOW);
        digitalWrite(leftdirpin, HIGH);
        digitalWrite(rightdirpin, LOW);
        analogWrite(leftpwmpin, 255-pwm-offset); // OFFSET LINE
        analogWrite(rightpwmpin, pwm);
        updateGyro();
    } else
    {
        // SENSING CASE
        stopMotors();
    }
}

```

```

    // Blue (B=HIGH) Stablizing
    digitalWrite(REDF, LOW);
    digitalWrite(GREEN, LOW);
    digitalWrite(BLUE, HIGH);
    delay(2500); // allows for sensor tower to stop wobbling
    // sense
    startMillis = millis(); // Record the start time when entering the else
loop

    while (millis() - startMillis < detectionDuration)
    {
        // Searching
        digitalWrite(REDF, LOW);
        digitalWrite(GREEN, HIGH);
        digitalWrite(BLUE, HIGH);
        if (checkDisturbances())
        {
            disturbanceDetected = true;
            flashSearchLights();
        }
        // Optional: Add a small delay to prevent overwhelming the loop
        delay(100);
    }
    currentleftencoderpulse = leftencoderpulse;
    currentrightencoderpulse = rightencoderpulse;
}
}
// Handle ToF sensor readings
if (millis() - lastToFRead >= 1000)
{
    readAllToF Sensors();
    lastToFRead = millis();
}
}

void stopMotors()
{
    digitalWrite(leftdirpin, HIGH);
    digitalWrite(rightdirpin, LOW);
}

```

```
    analogWrite(leftpwmpin, 255);
    analogWrite(rightpwmpin, 0);
}

void ALeadingLeft()
{
    if(digitalRead(leftencoderBpin) == LOW)
    {
        leftencoderpulse--;
    }
    else
    {
        leftencoderpulse++;
    }
}

void BLeadingLeft()
{
    if(digitalRead(leftencoderApin) == LOW)
    {
        leftencoderpulse++;
    }
    else
    {
        leftencoderpulse--;
    }
}

void ALeadingRight()
{
    if(digitalRead(rightencoderBpin) == LOW)
    {
        rightencoderpulse++;
    }
    else
    {
        rightencoderpulse--;
    }
}
```



```

void BLeadingRight()
{
    if(digitalRead(rightencoderApin) == LOW)
    {
        rightencoderpulse--;
    }
    else
    {
        rightencoderpulse++;
    }
}

void computePID()
{
    error = targetangle - yaw;
    eI = eI + error; // integral error
    eD = prev_error - error; // derivative error
    prev_error = error;

    // limit overflow of intergral error
    if (eI > 200)
    {
        eI = 0;
    }

    omega = Kp * error + Ki * eI + Kd * eD; // calculating rate of change of angle
(rad/s)

    if (omega >= 100) // limit max
    {
        omega = 100;
    }
    else if (omega <= 0) // limit min
    {
        omega = 0;
    }
}

void turnRight()
{
    vL = ((omega * wheelbase) / 2);
    vR = ((omega * wheelbase) / 2);
}

```

```

if(targetangle > yaw)
{
    digitalWrite(leftdirpin, HIGH);
    digitalWrite(rightdirpin, HIGH);
    analogWrite(leftpwmpin, 255-(vL*0.152)-45); // OFFSET LINE
    analogWrite(rightpwmpin, 255-(vR*0.152));
} else
{
    stopMotors();
    runonreturn = false;
}
}

int16_t readGyroZ()
{
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(0x47); // GYRO_ZOUT_H
    Wire.endTransmission(false);
    Wire.requestFrom(MPU_ADDR, 2, true);
    return Wire.read() << 8 | Wire.read();
}

void calibrateGyro()
{
    Serial.println("Calibrating gyro... Keep sensor stationary!");
    delay(1000);

    long total = 0;
    for(int i = 0; i < 1000; i++)
    {
        total += readGyroZ();
        delay(1);
    }
    gyroZOffset = total / 1000.0;
    Serial.print("Calibration offset: ");
    Serial.println(gyroZOffset);
}

void updateGyro()

```

```

{
    int16_t rawGyroZ = readGyroZ();
    float gyroZ = (rawGyroZ - gyroZOffset) / GYRO_SCALE;

    unsigned long currentTime = micros();
    float dt = (currentTime - prevTime) / 1000000.0;
    prevTime = currentTime;

    yaw -= gyroZ * dt;
}

/***** ToF Functions *****/
void initializeSensor(Adafruit_VL53L0X &sensor, int xshutPin, uint8_t newAddress,
const char* position)
{
    digitalWrite(xshutPin, LOW);
    delay(10);
    digitalWrite(xshutPin, HIGH);
    delay(10);
    if (!sensor.begin(newAddress))
    {
        Serial.print(position);
        Serial.println(" sensor failed to boot!");
        while(1);
    }
    Serial.print(position);
    Serial.print(" sensor initialized at 0x");
    Serial.println(newAddress, HEX);
}

int readSensor(Adafruit_VL53L0X &sensor, const char* position)
{
    VL53L0X_RangingMeasurementData_t measure;
    sensor.rangingTest(&measure, false);

    if(measure.RangeStatus != 4)
    {

```

```

        Serial.print(position);
        Serial.print(": ");
        Serial.print(measure.RangeMilliMeter);
        Serial.println(" mm");
        return(measure.RangeMilliMeter);
    } else
    {
        Serial.print(position);
        Serial.println(": Out of range");
        return(-1);
    }
}

void readAllToFSSensors()
{
    distanceFarRight = readSensor(loxFarRight, "Far Right");
    distanceMidRight = readSensor(loxMidRight, "Middle Right");
    distanceMidLeft = readSensor(loxMidLeft, "Middle Left");
    distanceFarLeft = readSensor(loxFarLeft, "Far Left");
}

bool checkFrontObstacle()
{
    VL53L0X_RangingMeasurementData_t measure;
    bool obstacleDetected = false;

    // Check first sensor
    loxFarLeft.rangingTest(&measure, false);
    obstacleDetected |= (measure.RangeMilliMeter <= 500 && measure.RangeStatus !=
4);

    // Check second sensor
    loxMidLeft.rangingTest(&measure, false);
    obstacleDetected |= (measure.RangeMilliMeter <= 500 && measure.RangeStatus !=
4);

    // Check third sensor
    loxMidRight.rangingTest(&measure, false);

```

```

    obstacleDetected |= (measure.RangeMilliMeter <= 500 && measure.RangeStatus !=
4);

    // Check fourth sensor
    loxFarRight.rangingTest(&measure, false);
    obstacleDetected |= (measure.RangeMilliMeter <= 500 && measure.RangeStatus !=
4);

    return obstacleDetected;
}

/***** Sound/Motion Functions *****/

bool checkDisturbances()
{
    // Read PIR sensors (true = motion detected)
    bool motionDetected = digitalRead(pirPin1) || digitalRead(pirPin2) ||
digitalRead(pirPin3) || digitalRead(pirPin4);

    // Read analog sound level (higher values = louder)
    bool soundDetected = analogRead(sensorAnalogPin) > loudNoiseThreshold;

    // Check with debounce timer
    if ((motionDetected || soundDetected) && (millis() - lastDisturbanceTime >
debounceTime))
    {
        lastDisturbanceTime = millis();
        disturbanceDetected = true;
        return true;
    }
    return false;
}

void handleIntruderDetection()
{
    Serial.println("Intruder detected!");
    digitalWrite(buzzerPin, HIGH);

```

```

    disturbanceDetected = true;
}

void flashSearchLights()
{
    static unsigned long lastFlashTime = 0;

    if (millis() - lastFlashTime >= beepInterval)
    {
        lastFlashTime = millis();
        flashState = !flashState;

        if (flashState)
        {
            // Red Buzzer ON
            digitalWrite(REDF, HIGH);
            digitalWrite(GREEN, LOW);
            digitalWrite(BLUE, LOW);
            digitalWrite(buzzerPin, HIGH);
        } else
        {
            // Buzzer Cyan
            digitalWrite(REDF, LOW);
            digitalWrite(GREEN, HIGH);
            digitalWrite(BLUE, HIGH);
            digitalWrite(buzzerPin, LOW);
        }
    }
}

///// IR
void calibrateSensor()
{
    // Auto-calibrate to ambient noise
    long sum = 0;
    for(int i = 0; i < 50; i++)
    {
        sum += analogRead(sensorAnalogPin);
        delay(20);
    }
}

```

```

    }
    loudNoiseThreshold = (sum / 50) + 400; // Set threshold 400 above average
    Serial.print("Calibrated sound threshold: ");
    Serial.println(loudNoiseThreshold);
}

void translateIR() {
    if (irrecv.decode()) {
        static unsigned long lastIRTime = 0;
        unsigned long currentTime = millis();

        // Debounce - ignore repeats within 200ms
        if (currentTime - lastIRTime > 200) {
            lastIRTime = currentTime;

            switch(irrecv.decodedIRData.decodedRawData) {
                case 0xBA45FF00: // POWER button
                    if (currentState == OFF) {
                        currentState = MOVING;
                    } else if (currentState == STANDBY) {
                        currentState = MOVING;
                    } else {
                        currentState = OFF;
                    }
                    break;

                case 0xB847FF00: // RESET button
                    currentState = STANDBY;
                    break;
            }
        }
    }
}

```